Using Helpers

Earlier we said it's OK to put code in templates. Now we're going to modify that statement. It's perfectly acceptable to put *some* code in templates—that's what makes them dynamic. However, it's poor style to put too much code in templates.

Three main reasons for this stand out. First, the more code you put in the view side of your application, the easier it is to let discipline slip and start adding application-level functionality to the template code. This is definitely poor form; you want to put application stuff in the controller and model layers so that it's available everywhere. This will pay off when you add new ways of viewing the application.

The second reason is that `html.erb` is basically HTML. When you edit it, you're editing an HTML file. If you have the luxury of having professional designers create your layouts, they'll want to work with HTML. Putting a bunch of Ruby code in there just makes it hard to work with.

The final reason is that code embedded in views is hard to test, whereas code split out into helper modules can be isolated and tested as individual units.

Rails provides a nice compromise in the form of helpers. A *helper* is simply a module containing methods that assist a view. Helper methods are output-centric. They exist to generate HTML (or XML, or JavaScript)—a helper extends the behavior of a template.

Your Own Helpers

By default, each controller gets its own helper module. Additionally, there's an application-wide helper named `application_helper.rb`. It won't be surprising to learn that Rails makes certain assumptions to help link the helpers into the controller and its views. While all view helpers are available to all controllers, it's often good practice to organize helpers. Helpers that are unique to the views associated with the `ProductController` tend to be placed in a helper module called